PulseWatch

Open-source uptime & status monitoring
that costs almost nothing to run

SYSTEM DESIGN + DESIGN SYSTEM + FEATURE ROADMAP

Version 1.0 | October 2026 | MIT licensed

Working name: **PulseWatch** (rename freely). The goal is a self-hostable alternative to UptimeRobot that matches its free tier and goes further: more monitor types, smarter alerts, SSL/domain expiry, status pages, and response-time analytics, all on a single small server.

Contents: 1 Product scope | 2 Features | 3 System design | 4 Data model | 5 API | 6 Cost plan | 7 Design system | 8 Screens | 9 Roadmap

1. Product scope

PulseWatch checks that your websites, APIs, servers and scheduled jobs are alive, tells you quickly when they are not, and shows the history in a clean dashboard and a public status page.

Principles

- **Cheap by default:** one process, one SQLite file, runs on a 1 vCPU / 512 MB VPS or a free-tier host.
- **Open source:** MIT license, no paid-only features, everything self-hostable with one Docker command.
- **Few false alarms:** confirm failures from a second location or retry before alerting.
- **Fast to use:** add a monitor in under 15 seconds; sensible defaults for everything else.

Target users

User	Need
Freelancers & agencies	Watch many client sites, share status pages, get alerts on phone.
Indie hackers & startups	Cheap uptime plus cron/heartbeat monitoring for background jobs.
Small teams	Shared dashboard, on-call alerts, incident history for post-mortems.

2. Feature list

Everything below is planned. The **Phase** column shows when it ships (see roadmap in section 9).

Area	Feature	Phase
Monitors	HTTP(S) with method, headers, body, auth, expected status	MVP
Monitors	Keyword present / absent check in response body	MVP
Monitors	Ping (ICMP), TCP port, UDP port	MVP
Monitors	DNS record check (A, AAAA, CNAME, MX, TXT) with expected value	v1
Monitors	Heartbeat / cron monitoring (your job pings PulseWatch)	v1
Monitors	JSON API assertion (JSONPath equals / contains / less-than)	v1
Monitors	SSL certificate expiry and chain validity	v1
Monitors	Domain (WHOIS/RDAP) expiry warnings	v1
Monitors	Port scan of common ports, SMTP/IMAP/DB connectivity	v2
Monitors	Multi-step transaction checks (login then fetch page)	v2
Monitors	Docker container / server agent (CPU, RAM, disk)	v2
Checks	Intervals from 30 s to 24 h; per-monitor timeout and retries	MVP
Checks	Confirm-before-alert (re-check N times or from 2nd region)	MVP
Checks	Maintenance windows (silence alerts, show on status page)	v1
Checks	Multi-region probes via small worker binary	v2
Alerts	Email (SMTP), Telegram, Slack, Discord, Microsoft Teams	MVP
Alerts	Generic webhook, ntfy push, Web Push (PWA notification)	v1
Alerts	WhatsApp / SMS via your own provider key (Twilio, Termii)	v1
Alerts	Escalation policy: notify A, then B after 10 min	v2
Alerts	Quiet hours, per-monitor routing, recovery notices, reminders	v1

Area	Feature	Phase
Analytics	Uptime % (24 h, 7 d, 30 d, 90 d), response-time chart, p50/p95	MVP
Analytics	Incident log with auto-generated duration and cause	MVP
Analytics	Weekly email report and CSV / JSON export	v1
Analytics	SLA report PDF for clients	v2
Status page	Public page per project, custom domain, dark mode, embed badge	v1
Status page	Manual incident posts and subscriber email updates	v1
Teams	Multiple users, roles (owner, editor, viewer), API keys	v1
Teams	Two-factor auth (TOTP), audit log, SSO (OIDC)	v2
Platform	REST API, Prometheus /metrics endpoint, Grafana-ready	v1
Platform	Installable PWA, i18n (English, French, Hausa, Yoruba, etc.)	v1
Platform	Import monitors from UptimeRobot / CSV; config-as-code (YAML)	v1
Platform	AI incident summary (optional, bring your own key)	v2

3. System design

Architecture overview

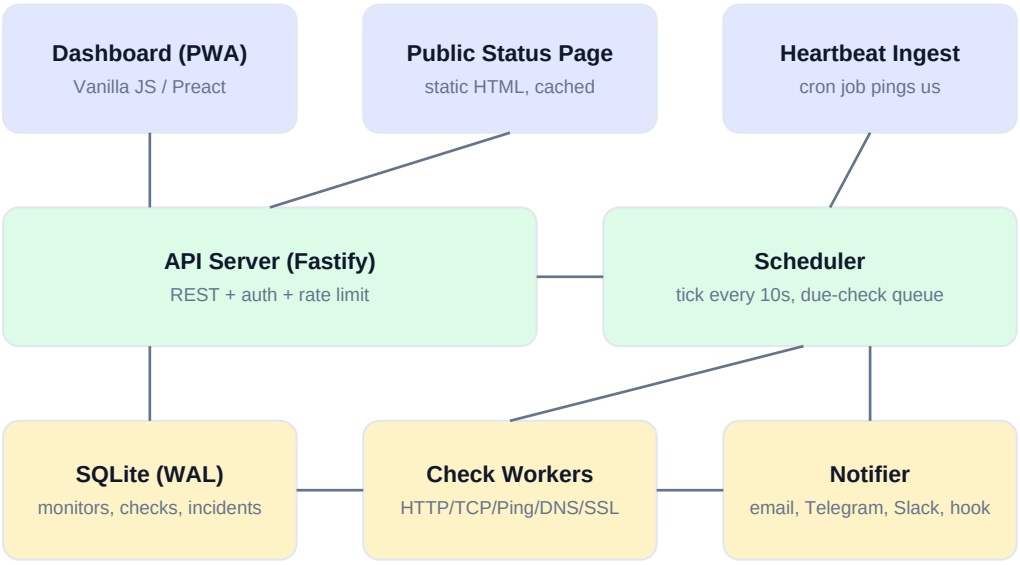

Figure 1. Single-node architecture. Every box runs inside one Node.js process (or one container) in the MVP.

Components

Component	Responsibility	Tech
API server	Auth, CRUD for monitors, alerts, status pages; serves dashboard and public pages.	Node 22, Fastify, Zod validation
Scheduler	Every 10 s selects monitors whose next_run_at is due and pushes them to the work queue.	In-process timer, SQL index on next_run_at
Check workers	Run the actual probe with timeout, record status, latency and error. Concurrency capped (e.g. 50).	undici (HTTP), net/tls, dns, ping
State machine	Applies retries and confirmation, opens or closes incidents, decides when to notify.	Pure functions, unit-tested
Notifier	Sends alerts through channels with retry and de-duplication.	nodemailer, fetch webhooks
Storage	Single SQLite file in WAL mode; rollups keep history small.	better-sqlite3
Dashboard	Mobile-first PWA; live updates over Server-Sent Events.	Preact + Vite, ~60 KB gzip
Status page	Pre-rendered HTML cached for 30 s; survives dashboard outage.	Server-rendered template

Check lifecycle

- Scheduler finds due monitors and sets `next_run_at = now + interval` before dispatch, so a crash never causes a storm.
- Worker probes and writes a row to `checks` (status, latency_ms, http_code, error).
- State machine evaluates: first failure sets state to *suspect* and re-checks after 20 s; second failure (or failure from another region) sets *down*.
- Going *down* opens an incident and queues notifications; recovery closes it and sends a recovery notice with total downtime.

- A nightly job rolls raw checks into hourly and daily aggregates, then deletes raw rows older than retention (default 30 days).

Reliability and scale

Concern	Approach
Monitor the monitor	Dead-man's-switch: PulseWatch pings an external free heartbeat (or a second instance) every minute; if it stops, you get alerted.
False positives	Retries + confirmation region + jitter; DNS or network failure on our side is detected by also probing a known-good host.
Notification delivery	Outbox table with retry and exponential backoff; each channel is tried independently.
Scaling past one node	Move SQLite to Postgres (adapter layer from day one), run extra stateless probe workers in other regions that report over HTTPS with a signed token.
Capacity (estimate)	A 1 vCPU box handles roughly 2,000 monitors at 1-minute interval; about 3 million raw check rows per month at that load, kept to ~250 MB with rollups.
Backups	Litestream streams the SQLite file to any S3-compatible bucket (Cloudflare R2 free tier); restore is one command.

Security

- Argon2id password hashing; optional TOTP; session cookies are HttpOnly + SameSite=Lax.
- **SSRF protection:** block checks to private ranges (127.0.0.0/8, 10/8, 169.254.169.254, etc.) unless the admin enables an allowlist. This matters because users can enter any URL.
- Secrets (SMTP, bot tokens) encrypted at rest with a key from an environment variable.
- Rate limits on login, API and heartbeat endpoints; signed webhook payloads (HMAC-SHA256).

4. Data model

Core tables (SQLite). Times are stored as UTC epoch milliseconds.

```
users(id, email, pass_hash, role, totp_secret, created_at)
projects(id, name, slug, owner_id, custom_domain, is_public)
monitors(id, project_id, name, type, target, interval_s, timeout_ms, retries,
         config_json, state, next_run_at, paused, created_at)
checks(id, monitor_id, ts, ok, latency_ms, code, error, region)         -- raw, 30 days
rollups(monitor_id, bucket_ts, granularity, up_count, down_count,
        avg_ms, p95_ms)                                                 -- hourly, daily
incidents(id, monitor_id, started_at, resolved_at, cause, auto)
channels(id, project_id, kind, config_enc, enabled)                     -- email, slack, ...
routes(monitor_id, channel_id, quiet_start, quiet_end, escalate_after_s)
outbox(id, channel_id, incident_id, payload, attempts, next_try_at, sent_at)
status_posts(id, project_id, title, body, kind, created_at)             -- manual incidents
maintenance(id, project_id, starts_at, ends_at, monitor_ids)
cert_info(monitor_id, issuer, valid_to, last_checked)
api_keys(id, user_id, hash, scope, last_used_at)
```

5. API surface

Method & path	Purpose
POST /api/auth/login	Session login (+ TOTP code if enabled)
GET / POST /api/monitors	List and create monitors
PATCH / DELETE /api/monitors/:id	Edit, pause, resume, delete
GET /api/monitors/:id/checks?range=24h	Time series for the chart

Method & path	Purpose
GET /api/incidents	Incident log with filters
POST /api/channels/:id/test	Send a test alert
GET /status/:slug	Public status page (HTML) and /status/:slug.json
GET /hb/:token	Heartbeat ping from cron jobs (also POST with payload)
GET /metrics	Prometheus-format metrics
GET /badge/:monitor.svg	Embeddable uptime badge
GET /api/events (SSE)	Live updates to the dashboard

6. Keeping it very cheap

The architecture avoids managed databases, queues and paid SaaS by default. Approximate monthly cost to run for yourself:

Setup	Hosting	Approx. cost / month
Personal / small team (up to ~500 monitors)	Oracle Cloud always-free ARM VM, or any 1 vCPU / 512 MB VPS	$0 to $5
Agency (up to ~2,000 monitors)	2 vCPU / 2 GB VPS + R2 backup bucket	$5 to $10
Extra probe regions (each)	Tiny VPS or free-tier container running the probe binary	$0 to $4
Alerts	Email via your SMTP or a free-tier provider; Telegram, Slack, Discord, ntfy are free	$0
Domain	One domain for dashboard and status pages	~$1 (amortised)

Prices vary by provider and change often; treat these as planning figures and confirm before committing.

Cost levers

- SQLite + rollups instead of Postgres + time-series DB.
- Single static Docker image (distroless Node), idle memory under 120 MB.
- Adaptive intervals: pages that have been stable for weeks can be checked less often if the user opts in.
- Optional hosted version later (to fund the project) while the self-hosted version stays complete.

Open-source plan

- MIT license, public GitHub repo, `docker compose up` quick start, one-page docs.
- Plugin interface for new monitor types and alert channels, so the community can add them without touching core.
- Conventional commits, CI on every pull request, good-first-issue labels, public roadmap.

7. Design system

A small, calm, status-first visual language. Colour is reserved for meaning (up, down, warning); everything else stays neutral so problems stand out immediately. Mobile-first, with full dark mode.

Colour tokens

Brand #4F46E5 | **Up** #16A34A | **Warning** #D97706 | **Down** #DC2626 | **Ink** #0F172A | **Muted** #64748B | **Surface** #F1F5F9

Token	Light	Dark	Use
--bg	#FFFFFF	#0B1120	Page background
--surface	#F1F5F9	#111A2E	Cards, table stripes
--border	#E2E8F0	#1E293B	Dividers, inputs
--text	#0F172A	#E2E8F0	Primary text (contrast 15:1 / 14:1)
--text-muted	#64748B	#94A3B8	Secondary text
--brand	#4F46E5	#818CF8	Buttons, links, focus ring
--up / --warn / --down	#16A34A / #D97706 / #DC2626	#4ADE80 / #FBBF24 / #F87171	Status only; always paired with an icon or label

Accessibility rule: status is never shown by colour alone. Up uses a check, warning a triangle, down a cross, plus the word.

Typography

Style	Font	Size / line	Weight
Display	Inter	28 / 34	700
Title	Inter	20 / 26	600
Heading	Inter	16 / 22	600
Body	Inter	14 / 20	400
Caption	Inter	12 / 16	500
Mono (URLs, codes)	JetBrains Mono	13 / 18	400

Inter and JetBrains Mono are open source (SIL OFL); self-host them to avoid third-party requests.

Spacing, shape, motion

Token	Value
Spacing scale (px)	4, 8, 12, 16, 24, 32, 48, 64
Radius	6 (inputs, chips), 10 (cards), 999 (pills)
Elevation	Borders first; one soft shadow for popovers: 0 8px 24px rgba(15,23,42,.12)
Breakpoints	sm 480, md 768, lg 1100; content max-width 1120
Motion	120 ms ease-out for hover/press, 200 ms for panels; respect prefers-reduced-motion
Icons	Lucide (MIT), 1.75 px stroke, 20 px default

Core components

Component	Variants & rules
Status pill	Up / Down / Paused / Suspect / Maintenance. Icon + label, 12 px caption, tinted background at 12% of the status colour.
Uptime bar	60 to 90 vertical ticks, one per time bucket; hover shows time, status, latency. Core signature element of the product.
Monitor card	Name, URL (mono, truncated), status pill, 24 h uptime, sparkline of latency, overflow menu.
Button	Primary (brand), Secondary (outline), Danger, Ghost. 40 px height, 44 px touch target on mobile.
Input / Select / Toggle	40 px height, visible label above, inline error text in --down with icon.
Chart	Line for latency, area for uptime; max 3 series; tooltip on tap or hover; table fallback for screen readers.
Toast & banner	Toast for transient confirmations; sticky banner for active incidents.
Empty state	Illustration-free: one sentence plus a primary action (for example, Add your first monitor).
Table	Sticky header, zebra off by default, row actions on the right, cards on mobile.

Uptime bar preview (signature component)

Mostly green, one red outage (two buckets) and one amber warning.

8. Key screens

Screen	Content	Mobile behaviour
Overview	Summary strip (monitors up / down / paused, 24 h uptime, open incidents), then monitor cards sorted down-first.	Single column, bottom tab bar: Overview, Incidents, Add, Settings.
Add monitor	Type picker, target, name, interval slider, alert routing. Advanced options collapsed. Test now button shows result before saving.	Full-screen sheet, 3 steps max.
Monitor detail	Status header, uptime bar, latency chart with range tabs (24 h, 7 d, 30 d, 90 d), SSL / domain expiry, recent checks, incidents.	Charts stack; range tabs scroll horizontally.
Incidents	Timeline with start, duration, cause, affected monitors; filter by project and date.	List with expandable rows.
Alert channels	List of channels with Test button and per-channel routing rules.	Same, with swipe to disable.
Status page editor	Choose monitors, grouping, logo, colours, custom domain, subscriber toggle; live preview.	Preview opens as a separate tab.
Public status page	Overall banner (All systems operational / Partial outage), per-service uptime bars, incident history, subscribe.	Fast, no JS required, under 50 KB.
Settings	Profile, 2FA, API keys, team, backups, retention, SMTP.	Grouped list.

Interaction rules

- Down monitors always float to the top; the page title and favicon change colour during an outage.
- Every destructive action needs confirmation with an undo toast where possible.
- Empty, loading and error states are designed for every list and chart (skeletons, not spinners).
- Keyboard: g o overview, n new monitor, / search.

9. Roadmap

Phase	Scope	Rough effort (1 developer)
MVP	HTTP / keyword / ping / TCP monitors, scheduler, state machine, email + Telegram + Slack + Discord alerts, dashboard, uptime and latency charts, incident log, Docker image.	3 to 4 weeks
v1	Status pages, SSL and domain expiry, heartbeat, DNS and JSON checks, maintenance windows, teams and API keys, webhooks, Web Push, weekly reports, import and export, PWA, Prometheus.	5 to 6 weeks
v2	Multi-region probes, escalation policies, transaction checks, server agent, SLA PDF, SSO, 2FA hardening, AI incident summary, plugin marketplace.	6 to 8 weeks

Suggested first steps

- Create the repo with the folder layout: `/server`, `/web`, `/probe`, `/docs`.
- Build the check engine and state machine first, with tests; it is the heart of the product.
- Ship the MVP to yourself, monitor your own sites for two weeks, then open the repo to the public.

> **Next step**
> I can now generate the actual MVP project: backend, scheduler, SQLite schema, alert channels and a dashboard following this design system, ready to run with one Docker command.